

ЛАБОРАТОРНАЯ РАБОТА №4

Агрегация данных, индексы и анализ производительности запросов MongoDB

Дисциплина: «Проектирование и применение NoSQL-технологий»
3 курс • MongoDB Compass / mongosh

1. Цель работы

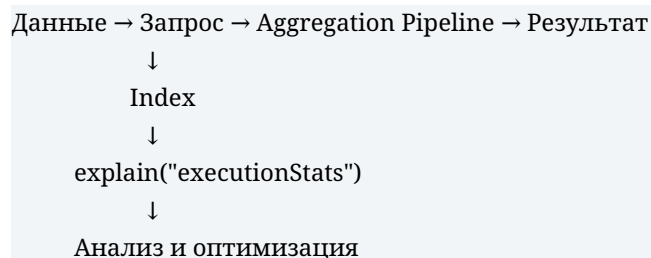
Получить практические навыки построения Aggregation Pipeline, создания простых и составных индексов, анализа планов выполнения запросов с помощью `explain()` и обоснованной оптимизации MongoDB-запросов на основе измеряемых показателей.

После выполнения работы студент должен уметь:

- использовать этапы `$match`, `$project`, `$group`, `$sort`, `$limit`, `$unwind` и `$lookup`;
- создавать простые и составные индексы;
- понимать различие COLLSCAN и IXSCAN;
- использовать `explain("executionStats")`;
- сравнивать `totalDocsExamined`, `totalKeysExamined` и `executionTimeMillis` до и после индексирования;
- объяснять порядок полей составного индекса с учетом принципа ESR (Equality-Sort-Range).

2. Постановка задачи

В предыдущих лабораторных работах были изучены CRUD-операции, проектирование документов, Embedding и Referencing. В лабораторной работе №4 необходимо перейти к обработке наборов документов и анализу производительности запросов.



3. Учебная предметная область

Базовый пример - интернет-магазин. Используются коллекции `products`, `customers` и `orders`. Студент по своему варианту создает аналогичную модель.

```
use ecommerce_lab4
```

```
db.createCollection("products")
db.createCollection("customers")
db.createCollection("orders")
```

4. Подготовка тестовых данных

Добавьте минимум 30 документов в основную коллекцию. Данные должны различаться по категориям, цене, рейтингу, количеству на складе и другим полям, чтобы результаты агрегации были содержательными.

```
db.products.insertMany([
  {productId:101,name:"Laptop Lenovo",category:"Notebook",price:420000,stock:15,rating:4.7,tags:
  ["office","programming"]},
  {productId:102,name:"Laptop ASUS",category:"Notebook",price:520000,stock:8,rating:4.8,tags:
  ["gaming","programming"]},
  {productId:103,name:"Phone Samsung",category:"Smartphone",price:310000,stock:25,rating:4.6,tags:
  ["mobile","android"]},
  {productId:104,name:"Phone Apple",category:"Smartphone",price:650000,stock:12,rating:4.9,tags:["mobile","ios"]}
])
```

5. Задание 1. Фильтрация с \$match

\$match фильтрует документы и передает дальше только подходящие элементы. По смыслу этап похож на фильтр find(), но используется внутри конвейера агрегации.

```
db.products.aggregate([
  {$match:{category:"Notebook",price:{$lte:500000}}}
])
```

Самостоятельно выполните:

фильтрацию по одному строковому полю;

фильтрацию по диапазону числовых значений;

фильтрацию одновременно по двум условиям;

фильтрацию с \$or или \$in.

6. Задание 2. Формирование результата с \$project

\$project позволяет определить, какие поля попадут в результат, а также создавать вычисляемые поля.

```
db.products.aggregate([
  {$match:{category:"Notebook"}},
  {$project:{_id:0,name:1,price:1,rating:1}}
])
```

Создайте pipeline, который возвращает только 3-4 нужных поля. Затем создайте вычисляемое поле, например стоимость остатка товара:

```
db.products.aggregate([
  {$project:{_id:0,name:1,price:1,stock:1,inventoryValue:{$multiply:["$price","$stock"]}}}
])
```

7. Задание 3. Группировка с \$group

\$group объединяет документы по ключу и позволяет вычислять агрегированные показатели. Поле _id внутри \$group задает ключ группировки.

```
db.products.aggregate([
  {$group:{
    _id:"$category",
    productCount:{$sum:1},
    avgPrice:{$avg:"$price"},
    minPrice:{$min:"$price"},
    maxPrice:{$max:"$price"}
  }}
])
```

Самостоятельно рассчитайте по группам: количество документов, среднее значение, сумму, минимум и максимум.

8. Задание 4. Сортировка и ограничение результата

```
db.products.aggregate([
  {$match:{stock:{$gt:0}}},
  {$sort:{price:-1}},
  {$limit:5},
  {$project:{_id:0,name:1,price:1,stock:1}}
])
```

Получите TOP-5 документов по выбранному числовому показателю. Объясните направление сортировки: 1 - по возрастанию, -1 - по убыванию.

9. Задание 5. Работа с массивами через \$unwind

\$unwind разворачивает массив: для каждого элемента массива формируется отдельный потоковый документ.

```
db.products.aggregate([
  {$unwind:"$tags"},
  {$group:{_id:"$tags",count:{$sum:1}}},
  {$sort:{count:-1}}
])
```

Создайте собственный запрос, который разворачивает массив и определяет наиболее часто встречающиеся значения.

10. Задание 6. Объединение коллекций через \$lookup

\$lookup позволяет выполнить левое внешнее соединение с другой коллекцией. Для примера создайте клиентов и заказы.

```
db.customers.insertMany([
  {customerId:501,name:"Aidar"},
  {customerId:502,name:"Dana"}
])

db.orders.insertMany([
  {orderId:"ORD-1",customerId:501,total:420000,status:"paid"},
  {orderId:"ORD-2",customerId:501,total:25000,status:"paid"},
  {orderId:"ORD-3",customerId:502,total:310000,status:"new"}
])

db.orders.aggregate([
  {$lookup:{
    from:"customers",
    localField:"customerId",
    foreignField:"customerId",
    as:"customer"
  }},
  {$unwind:"$customer"},
  {$project:{_id:0,orderId:1,total:1,"customer.name":1}}
])
```

Самостоятельно выполните \$lookup между двумя коллекциями своего варианта и объясните поля from, localField, foreignField и as.

11. Задание 7. Многоэтапный Aggregation Pipeline

Создайте pipeline минимум из четырех этапов. Например, выберите доступные товары, сгруппируйте по категории, вычислите среднюю цену, отсортируйте категории и ограничьте результат.

```
db.products.aggregate([
  {$match:{stock:{$gt:0}}},
  {$group:{_id:"$category",avgPrice:{$avg:"$price"},count:{$sum:1}}},
  {$sort:{avgPrice:-1}},
  {$limit:5}
])
```

12. Задание 8. Анализ запроса без индекса

Выполните запрос по полю, для которого еще не создан пользовательский индекс, и получите статистику выполнения.

```
db.products.find({category:"Notebook"}).explain("executionStats")
```

Зафиксируйте в отчете: winningPlan/stage, наличие COLLSCAN или IXSCAN, nReturned, totalDocsExamined, totalKeysExamined и executionTimeMillis. Значения зависят от набора данных и среды, поэтому их необходимо получить экспериментально.

13. Задание 9. Создание простого индекса

```
db.products.createIndex({category:1})
```

Повторите тот же explain("executionStats") и сравните показатели до и после создания индекса. Сделайте вывод, уменьшилось ли число просмотренных документов и изменился ли план выполнения.

14. Задание 10. Составной индекс

Предположим, приложение часто фильтрует товары по категории и сортирует их по цене:

```
db.products.find({category:"Notebook"}).sort({price:1})
```

Создайте составной индекс:

```
db.products.createIndex({category:1,price:1})
```

Порядок полей важен. В рамках принципа ESR сначала обычно рассматриваются поля равенства (Equality), затем сортировки (Sort), затем диапазона (Range), если такой порядок соответствует реальным запросам.

15. Задание 11. Проверка составного индекса

```
db.products.find({category:"Notebook"}).sort({price:1}).explain("executionStats")
```

Проанализируйте, использован ли IXSCAN, требуется ли отдельная стадия SORT, сколько ключей и документов просмотрено. Сравните с вариантом без подходящего составного индекса.

16. Задание 12. Эксперимент с ESR

Выполните запрос с равенством, сортировкой и диапазоном. Например:

```
db.products.find({category:"Notebook",rating:{$gte:4.5}}).sort({price:1})
```

Сравните два возможных индекса, например {category:1,price:1,rating:1} и {category:1,rating:1,price:1}. Не утверждайте заранее, какой вариант быстрее: используйте

explain("executionStats") и объясните наблюдаемый результат с учетом формы запроса и принципа ESR.

17. Обязательное сравнительное исследование

Показатель	До индекса	После индекса
План (COLLSCAN/IXSCAN)		
nReturned		
totalDocsExamined		
totalKeysExamined		
executionTimeMillis		

Заполните таблицу реальными значениями из своей среды. Для небольшого учебного набора executionTimeMillis может отличаться незначительно; более надежным признаком работы индекса часто является изменение плана и количества просмотренных документов/ключей.

18. Индивидуальное задание

Для своего варианта необходимо создать минимум 3 коллекции и 30 документов в основной коллекции. Выполнить минимум 10 Aggregation Pipeline запросов и 4 эксперимента с индексами.

№	Предметная область	Основная коллекция	Поля для анализа
1	Интернет-магазин	products	category, price, rating, stock
2	Университет	students	group, gpa, year, skills
3	Социальная сеть	posts	authorId, createdAt, likes, tags
4	Доставка еды	orders	restaurantId, status, total, createdAt
5	Онлайн-библиотека	books	genre, authorId, rating, year
6	Гостиница	rooms	hotelId, type, price, status
7	Медицинская система	appointments	doctorId, patientId, date, status
8	Такси	trips	driverId, status, fare, createdAt
9	Онлайн-кинотеатр	movies	genre, year, rating, duration
10	Умный дом	measurements	deviceId, type, value, timestamp
11	Мониторинг транспорта	events	vehicleId, type, timestamp, routeId

12	Онлайн-обучение	courses	teacherId, category, rating, price
13	Склад	products	category, quantity, warehouseId, price
14	Музыкальный сервис	tracks	artistId, genre, duration, plays
15	Новостной портал	articles	category, authorId, views, publishedAt
16	Вакансии	vacancies	companyId, city, salary, skills
17	Управление проектами	tasks	projectId, status, priority, dueDate
18	Авиабилеты	flights	from, to, price, departure
19	Каталог смартфонов	smartphones	brand, price, rating, ram
20	Банковский мониторинг	transactions	accountId, type, amount, timestamp

19. Обязательные задания по своему варианту

Выполнить 3 запроса с \$match.

Выполнить 2 запроса с \$project, один с вычисляемым полем.

Выполнить минимум 2 группировки с \$group.

Использовать \$sum, \$avg, \$min или \$max.

Использовать \$sort и \$limit.

Использовать \$unwind для массива.

Использовать \$lookup между двумя коллекциями.

Создать pipeline минимум из 4 этапов.

Получить explain("executionStats") до индексирования.

Создать простой индекс и повторить измерение.

Создать составной индекс минимум по двум полям.

Выполнить эксперимент с порядком полей составного индекса.

Сравнить COLLSCAN и IXSCAN.

Заполнить таблицу статистики до/после.

Сформулировать вывод о пользе и стоимости индексов.

20. Требования к отчету

- титульный лист и цель работы;

- номер варианта и описание предметной области;
- описание структуры коллекций;
- примеры тестовых документов;
- не менее 10 Aggregation Pipeline запросов с результатами;
- пример \$lookup и \$unwind;
- список созданных индексов;
- не менее 4 explain("executionStats");
- сравнительная таблица до/после индексирования;
- объяснение ESR на собственном запросе;
- скриншоты MongoDB Compass или mongosh;
- итоговый вывод.

21. Критерии оценивания

Критерий	Баллы
Подготовка данных и коллекций	10
Использование \$match и \$project	10
Группировка и агрегатные операторы	15
\$sort, \$limit, \$unwind	10
\$lookup	10
Многоэтапный pipeline	10
Простой индекс	5
Составной индекс и ESR	10
Анализ explain()	10
Сравнение производительности и вывод	5
Защита работы	5
ИТОГО	100

22. Контрольные вопросы

1. Что такое Aggregation Pipeline?
2. Для чего используется \$match?
3. Чем \$project отличается от обычной фильтрации?
4. Для чего применяется \$group?
5. Что означает поле _id внутри \$group?
6. Для чего используются \$sum, \$avg, \$min и \$max?
7. Что делает \$unwind?

8. Для чего применяется \$lookup?
9. Что такое индекс MongoDB?
10. Чем простой индекс отличается от составного?
11. Что означает COLLSCAN?
12. Что означает IXSCAN?
13. Для чего используется explain("executionStats")?
14. Что показывают totalDocsExamined и totalKeysExamined?
15. Почему индекс может ускорять чтение, но увеличивать стоимость записи?
16. Почему порядок полей составного индекса важен?
17. Что означает принцип ESR?
18. Почему нельзя оценивать эффективность индекса только по executionTimeMillis на маленьком наборе данных?
19. Как Access Patterns связаны с проектированием индексов?
20. В каких случаях индекс может оказаться бесполезным?

23. Рекомендуемые источники

- MongoDB Documentation. Aggregation Operations.
<https://www.mongodb.com/docs/manual/aggregation/>
- MongoDB Documentation. Aggregation Pipeline Stages.
<https://www.mongodb.com/docs/manual/reference/operator/aggregation-pipeline/>
- MongoDB Documentation. Indexes. <https://www.mongodb.com/docs/manual/indexes/>
- MongoDB Documentation. Explain Results.
<https://www.mongodb.com/docs/manual/reference/explain-results/>
- MongoDB Documentation. The ESR (Equality, Sort, Range) Guideline.
<https://www.mongodb.com/docs/manual/tutorial/equality-sort-range-guideline/>
- MongoDB University. <https://learn.mongodb.com/>